Midterm Exam, Faculty of Engineering, Chulalongkorn University
Course ID: 2110215 Course Name: Programming Methodology I
Second Semester, Date: 6 March 2020 Time: 8.30-11.30AM

Name ..... Student ID. ..... No. in CR .....

Instructions

1. Write your Student ID, full name, and your number in CR58 in the space provided on this page.
2. Your answer must be on the computer center's machine in front of you.
3. Documents and files are allowed inside the exam room; however, internet and flash drive are prohibited. Borrowing is not allowed unless it is supervised by the proctor.
4. You must not carry mobile phone and flash drive during the exam.
5. ***** You must not bring any part of this exam paper outside. The exam paper is a government's property. Violators will be prosecuted under a criminal court and will receive an F in the subject. *****
6. Students who wish to leave the exam room before the end of the exam period, must raise their hands and ask for permission before leaving the room. Students must leave the room in the orderly manner.
7. Once the time expires, student must stop typing and must remain seated quietly until the proctors collect all the exam papers or given exam booklets. Only then, the students will be allowed to leave the room in an orderly manner.
8. Any student who does not obey the regulations listed above will receive punishment under the Faculty of Engineering Official Announcement on January 6, 2003 regarding the exam regulations.
    a. With implicit evidence or showing intention for cheating, student will receive an F in that subject and will receive an academic suspension for 1 semester.
    b. With explicit evidence for cheating, student will receive an F in that subject and will receive an academic suspension for 1 year.

Please sign and submit

Signature (.....)

Important Rules

- You must not bring any part of this exam paper outside. The exam paper is a government's property. **Violators will be prosecuted under a criminal court and will receive an F in the subject.**
- It is a student's responsibility to check the file. If it is corrupted or cannot be open, there is no score.
- For each question, a table is given, showing (color-coded) whether or not you have to modify or create each method or variable.

** Noted that Access Modifier Notations can be listed below*

+ (public)
# (protected)
- (private)
<u>Underline</u> (static)
Italic (abstract)

Set-Up Instruction

- Set workspace to "**C:\temp\progmeth2019_2\Midterm_2110215_(your id)_(FirstName)**" (if not exist, you must create it)
  - For example, "C:\temp\progmeth2019_2\Midterm_2110215_6170156021_Boss"
- All your files must be in the workspace.
- **The code outside of the workspace will not be collected, or graded**

Scoring (Total 30 points, will be scaled to 40 points)

- Part1 = 10 points
- Part2 = 10 points
- Part3 = 10 points

Part 3: Abstract Classes

1. Objective

1) Be able to implement abstract classes and use abstract methods.

2. Instruction

1) Create Java Project named "**2110215_Midterm_Part3**".
2) Copy all folders in **"toStudent/Part3"** to your project directory src folder.
3) You are to implement the following classes (detail for each class is given in section 3 and 4)

   a) **Employee** (package logic)
   b) **OfficeWorker** (package logic)
   c) **Janitor** (package logic)
   d) **Database** (package logic)
4) Make UML class diagram for classes in package logic, using ObjectAid.
5) **JUnit for testing is in package test.grader**

3. Problem Statement: Macro Cosmos

You are a part of World-Leader Mega-Corporate Company, Macro Cosmos. The company is developing its new Employee Management System. Unfortunately, the previous programmer went to Wuhan for vacation then unable to comeback until a few weeks later. However, the system needs to be finished by this week. So, you are tasked to complete the system.

4. Implementation Detail

The class package is summarized below.

*** In the following class description, only details of IMPORTANT fields and methods are given. ***

4.1 Package logic

4.1.1 class BackEndAPI

This class is the back-end API that the previous programmer has completed. You don't have to implement anything in this class. However, you might need to use the provided methods later.

Method

Name	Description
+ int calculateMonthlySalary(int baseSalary,int bonus, int day)	Calculate monthly salary of the employee.
+ getOfficeWorkerDescription(int id,String name,String department,int bonus)	Format the Description String of the Office Worker.
+ getJanitorDescription(int id,String name,String area,int bonus)	Format the Description String of the Janitor.

4.1.2 abstract class Employee **/* You must implement this class from scratch */**

This abstract class is a base class for all types of Employee. It contains the basic information and methods that all the Employee should have.

Field

Name	Description
# String name	Name of the employee
# int id	ID of the employee
# int baseSalary	Base salary of the employee per working hour
# int bonus	Bonus of the employee

Method

Name	Description
+ Employee(String name, int id,int baseSalary)	This is the Constructor. This sets all the fields with their respective value. (For bonus, set it to 0)
+ *int computeSalary()*	Compute monthly salary of the respective Employee. Exact implementations are different between each type of employees.
+ *String getDescription()*	Obtain the description of the respective Employee Exact details are different between each type of employees.
+ getter-setter for all fields.	**For setBaseSalary and setBonus, if the value is below 0, set it to 0.**

4.1.3. class OfficeWorker **/* You must implement this class from scratch */**

This class is a type of Employee who works in the office of each department of the company. Each employee starts with base salary of 30$ per hour and works 20 days each month.

Fields

Name	Description
- String department	Department of the OfficeWorker

Method

Name	Description
+ OfficeWorker(String name, int id,String department)	This is the Constructor. In addition to setting the information of the super class, it also sets the department, too. The salary of the Office Worker is 30$ per hour.

+ int computeSalary()	This method returns monthly salary of the Office Worker. You need to obtain the calculated value from a method in BackEndAPI. OfficeWorker works 20 days each month.
+ String getDescription()	This method returns the description of the Office Worker. The String format is provided in BackEndAPI.
getter/setter for department	

4.1.4. class Janitor **/* You must implement this class from scratch */**

This class is a type of Employee who cleans the floor of the respective area in the company. Each Janitor starts with base salary of 15$ per hour and works 30 days each month.

Fields

Name	Description
- String area	The Area where Janitor works at

Method

Name	Description
+ Janitor(String name, int id,String area)	This is the Constructor. In addition to setting the information of the super class, it also sets the area, too. The salary of the Janitor is 15$ per hour.
+ int computeSalary()	This method returns monthly salary of the Janitor. You need to obtain the calculated value from a method in BackEndAPI. Janitor works 30 days each month.

+ String getDescription()	This method returns the description of the Janitor. The String format is provided in BackEndAPI.
getter/setter for area	

4.1.5. class Database /* PARTIALLY PROVIDED */

This class is the employee database. It contains all employee information, as well as the all relevant methods.

Fields

Name	Description
- ArrayList<Employee> employees	ArrayList containing all employees of the company

Method

Name	Description
+ Database()	This is the Constructor. Initialize the employees Array List.
+ Employee getEmployeeById(int id)	This method returns an Employee from the ArrayList who has the matched id. If there is no match, returns null.
+ Employee getEmployeeByIndex(int index)	This method returns an Employee at the provided index.
+ boolean addEmployee(Employee e)	This method adds new Employee into the ArrayList. If the employee with the same ID already existed, then returns false (and do not add). Otherwise, add the employee and return true.
+ boolean removeEmployeeById(int id)	This method removes an Employee with the provided ID from the ArrayList. If the employee with the same ID does not exist, then do nothing and return false. Otherwise, remove the employee and return true.

+ ArrayList<String> getAllEmployeeDescriptions()	/* FILL CODES */ This method returns an ArrayList containing the description of all employees.
+ int calculateAllSalary()	/* FILL CODES */ This method returns the summation of the monthly salary from all employees.
+ int getTotalEmployeeCount()	This method returns the current employee counts in the database.

4.2 Package main

4.2.1 class Main

This class is the main program. You don't have to implement anything in this class. You can test the program by running this class.

5. Finished Code Run Example

5.1.1. Adding an Employee

```
What would you like to do today?
1. Add Employee
2. Remove Employee
3. View Employee List
4. Manage Employee Bonus
5. Calculate Total Salary
> 1
================================
Adding new Employee Entry.
Please input the name.
> Bede
Please input the id.
> 908
Which type of the employee are they?
1) Office Worker
2) Janitor
> 2
================================
Please input the working area.
> Ballonlea Gym Locker Room
908 Bede [Janitor at Ballonlea Gym Locker Room] [Bonus: 0$] has been added to the employee database!
```

5.1.2. Adding an Employee with the same ID

```
=================================
What would you like to do today?
1. Add Employee
2. Remove Employee
3. View Employee List
4. Manage Employee Bonus
5. Calculate Total Salary
> 1
================================
Adding new Employee Entry.
Please input the name.
> Mark
Please input the id.
> 11034
Which type of the employee are they?
1) Office Worker
2) Janitor
> 1
================================
Please input the department.
> MC Entertainment
Error Adding Employee ID.11034
Employee Record with the same ID already exists.
```

5.2.1 Removing an Employee

```
=================================
What would you like to do today?
1. Add Employee
2. Remove Employee
3. View Employee List
4. Manage Employee Bonus
5. Calculate Total Salary
> 2
================================
Removing Employee Entry.
Please input the id of the employee to be removed.
> 11034
Removing Employee ID.11034 Successfully.
```

5.2.2 Removing an Employee that does not exist

```
What would you like to do today?
1. Add Employee
2. Remove Employee
3. View Employee List
4. Manage Employee Bonus
5. Calculate Total Salary
> 2
================================
Removing Employee Entry.
Please input the id of the employee to be removed.
> 123
Error Removing Employee ID.123
Employee Record does not exist.
```

5.3 View Employee Lists

```
What would you like to do today?
1. Add Employee
2. Remove Employee
3. View Employee List
4. Manage Employee Bonus
5. Calculate Total Salary
> 3
================================
List of all Employee.
1) 11035 Kevin [Office Worker in MC Air Department] [Bonus: 0$]
2) 231 Jane [Janitor at Hotel Ionia Main Entrance] [Bonus: 0$]
3) 232 Carla [Janitor at Hammerlocke Stadium Corridor] [Bonus: 0$]
4) 908 Bede [Janitor at Ballonlea Gym Locker Room] [Bonus: 0$]
```

5.4 Manage Employee Bonus

```
What would you like to do today?
1. Add Employee
2. Remove Employee
3. View Employee List
4. Manage Employee Bonus
5. Calculate Total Salary
> 4
================================
Employee Bonus Management.
Which employee bonus should be tweaked?
1) 11035 Kevin [Office Worker in MC Air Department] [Bonus: 0$]
2) 231 Jane [Janitor at Hotel Ionia Main Entrance] [Bonus: 0$]
3) 232 Carla [Janitor at Hammerlocke Stadium Corridor] [Bonus: 0$]
4) 908 Bede [Janitor at Ballonlea Gym Locker Room] [Bonus: 0$]
===================
> 1
11035 Kevin
Base Salary: 30$/hrs
Current Bonus 0$
===================
How much the bonus be changed?
>
300
Bonus of the employee No.11035 Kevin has been changed to 300$
```

5.5 Calculate Total Salary

```
What would you like to do today?
1. Add Employee
2. Remove Employee
3. View Employee List
4. Manage Employee Bonus
5. Calculate Total Salary
> 5
================================
The total salary of the company for this month is
15900$
--------------------------------
```

Final Exam, Faculty of Engineering, Chulalongkorn University
Course ID: 2110215 Course Name: Programming Methodology I
Second Semester, Date: 11 May 2020 Time: 13.00-16.00 (Online Exam)

Instructions

1. Your projects must be submitted as an assignment on MyCourseVille (please see "How to Submit").
2. Documents and files are allowed. Internet search is allowed (^_^).
3. No communication with another person is allowed.
4. Student must submit files before the time expires.
5. Any student who does not obey the regulations listed below will receive punishment under the Faculty of Engineering Official Announcement on January 6, 2003 regarding the exam regulations.
    a. With implicit evidence or showing intention for cheating, student will receive an F in that subject and will receive an academic suspension for 1 semester. (**หมายความว่า แค่ "สงสัย จากตัวโค้ดที่ทำ" ว่าลอกกัน อาจารย์มีสิทธิให้ F และพักการเรียนได้เลย**)
    b. With explicit evidence for cheating, student will receive an F in that subject and will receive an academic suspension for 1 year. (**หมายความว่า ถ้าเห็นว่ากำลังลอกเพื่อนอยู่ตอนนั้น ก็ F และพักการเรียน 1 ปี**)

Important Rules

- It is a student's responsibility to check the file. If it is corrupted or cannot be open, there is no score.

** Noted that Access Modifier Notations can be listed below*

+ (public)

(protected)

- (private)

<u>Underline</u> (static)

Italic (abstract)

Set-Up Instruction

- Set workspace to your usual workspace.
- All your exam files for each question must be in the project folder for that question.

How to Submit

There are 3 questions (hence 3 projects) in this exam.

- In your workspace directory, zip all 3 projects' folders into a zip file called yourID.zip (for example, 6233001121.zip).
- Submit the zip file as an assignment on MyCourseville.
- Make sure you do not include other files apart from files for each question. **A project that is too big in size may result in your submission NOT getting saved**.

Scoring (Total 30 points, will be scaled to 25 points)

- Part1 = 10 points
- Part2 = 9 points
- Part3 = 11 points

Part 1: Interface (Guildmaster's Revenge)

1. Objectives

1) Students are able to implement interfaces and use them with classes.

2. Instructions

1) Create Java Project named **"2110215_2_Final_Part1"** in your workspace.
2) Copy all folders in **"toStudent/Part1"** to your project directory src folder.
3) You are to implement the following classes (details for each class are given in section 4.)
    a) Tank (package fighters.derived)
    b) Wizard (package fighters.derived)
    c) Guildmaster (package fighters.derived)
4) After creating all the required classes, you are required to draw a UML Diagram using ObjectAid. Save it as .png in the root of the project's folder. **(The UML Diagram is worth 2 points.)**

3. Problem Statement: Foe Emblem: Guildmaster's Revenge

1. Problem Statement: Guild Member Database

You have been transported into a fantasy world by a magical truck, and was found by a desperate guildmaster who does not know how to use computers. As the only person in this fantasy world who knows how to use a computer, you are asked to implement a system that allows him to manage guild data, including member information and department information.

The program example is shown below. The program should be run from the Main class in the package main.

```
===== Welcome to the Guild Member Database! =====
===== MAIN MENU =====
0) View Departments and Members
1) New Department
2) Remove Department
3) New Member
4) Remove Member
5) Exit
```

The guildmaster from Lab1 has betrayed you and claimed to have written the code by himself. You decide to teach him a lesson. Nobody gets away with taking credit for a software you wrote with your own sweat and blood. However, the Guildmaster has become too powerful to deal with on your own. You need help from a few old friends you met along the way.

1) The wizard: The wizard can shoot piercing magic bolts that pierces through defense from a distance but cannot raise up any shields.
2) The tank: He does not attack, but he has a very strong shield that can block powerful attacks.

```
==Fight the Guildmaster==
0123456789
wt-------G
Wizard: 20/20
Tank: 30/30
Guildmaster: 50/50

It is now Wizard's turn.

Choose your Action:
1) Move
2) Attack
3) Guard
4) Wait
```

You are to implement a battle strategy planner. Your program will be able to do the following:

1) Allow each character to take their turns. For each of your allies, you will be able to choose their actions. However, not all characters are able to do everything. For example, Tank cannot attack and Wizard cannot guard. The actions are as follow:
    a. Move: Move to the right (positive, +) or the left (negative, -).
    b. Attack: Deal damage to a character within this unit's range.
    c. Guard: Raise a shield. Units who can guard have a certain amount of defense that already deducts incoming damage. Raising a shield doubles this defense. A unit who attacks or moves stops guarding.
    d. Wait: Do nothing. If the unit was previously on guard, then the unit continues to be on guard until they move or attack.
2) Allow Guildmaster to automatically fight as an enemy. The Guildmaster's strategy is very simple.
    a. If there is a space directly to the left, the Guildmaster moves left.
    b. If he cannot move to the left, the Guildmaster assumes that there is an enemy to the left and attacks it.
    c. If Guildmaster arrives at cell 0, the battle is lost, and the game terminates.
    d. If Guildmaster's HP reaches 0, the battle is won, and the game terminates.
3) Display every character's position on the map.
    a. The map is displayed as a one-dimensional line with 10 cells.
    b. Each empty cell is represented as a "-".
    c. A cell occupied by a character will be represented as an alphabet. Lowercase for allies and uppercase for enemies.
    d. At the start of the game, the map will look like this:

```
0123456789
wt-------G
```

Meaning that at the start of the game, the wizard is at cell 0, the tank is at cell 1, and the guildmaster is at cell 9.

The program can be run from the class Application.java in the package app.

4. Implementation Detail

The class package is summarized below. Note that only important method that you need to complete AND provided methods that might be useful in completing this application are listed.

- private

protected

+ public

Static variables and methods are <u>underlined</u>.

4.1 Package fighters.base

ALREADY GIVEN

Everything in this package is already written for you. Only important variables and methods will be listed for you to use.

4.1.1. public abstract class Unit

Variables

Name	Description
# String name	What this unit is called.
# String symbol	The symbol used to represent this unit in the map.
# int maxHealth	The amount of health this unit is spawned with.
# int health	The current amount of health. When it reaches 0, the unit is defeated and is removed from the map.
# int speed	How many steps this unit can move per turn.
# int power	The amount of damage dealt when attacking.
# int range	How far this unit can attack.
# int defense	How much damage this unit can deduct.
# boolean onGuard	Whether or not this unit is on guard.
# int location	Where the unit is currently standing on the map.

# boolean playerControlled	If true, then the player chooses the unit's actions. If false, the unit will be controlled by a simple AI.

Constructor

Name	Description
+ Unit(String name, String symbol, int maxHealth, int speed, int location, boolean playerControlled)	Constructor method. All the following variables are set: Health is set to Max Health. Power, range, and defense are set to 0. onGuard is set to false.

Methods

Name	Description
+ boolean move (int spaces)	Attempts to move to the right (if positive) or the left (if negative). Cannot move into a space occupied by another unit. Cannot move into a space blocked by a unit controlled by an opposing team. Cannot move out of the map. Cannot move 0 spaces. If the move is successful, return true. Otherwise return false.
+ boolean sameTeam (Unit unit)	Returns true if this unit and the given unit's playerControlled are the same value. Returns false otherwise.
+ getter/setter for all variables	Note that by default getters for all ints and Strings start with get… and getters for Booleans start with is…

	For example: getName(), getMaxHealth(), isPlayerControlled()

4.1.2. public interface Attackable

Variables

None.

Methods

Name	Description
+ abstract int attack (Unit e)	Attacks the given unit. Returns the damage dealt, depending on each unit's damage calculation formula.

4.1.3. public interface Guardable

Variables

None

Methods

Name	Description
+ abstract void guard()	Calls an action to guard. Depending on the unit, guarding for each unit may be different.

4.2 package fighters.derived

TO BE IMPLEMENTED

This entire package must be written from scratch.

4.2.1 public class **Tank**

This class is a unit that can guard, but not attack.

Variables

None.

Constructor

Name	Description
+ Tank(int maxHealth, int speed, int defense, int location)	Initializes the values for the tank. For the values not given in the constructor, they are fixed as follow: Tank's name is always "Tank". Tank's symbol is always "t" (lowercase). Tank's playerControlled is always true.

Methods

Name	Description
+ void guard()	The tank gets on guard and onGuard is set as true.
+ boolean move(int spaces)	When the tank moves, he is no longer on guard. Set onGuard as false, then call the move function from Unit.

4.2.2 public class **Wizard**

This class is a unit that can perform ranged attacks that ignore the target's defense, but cannot guard.

Variables

None.

Constructor

Name	Description
+ Wizard(int maxHealth, int speed, int power, int location)	Initializes the values for the wizard. For the values not given in the constructor, they are fixed as follow: Wizard's name is always "Wizard". Wizard's symbol is always "w" (lowercase). Wizard's playerControlled is always true. Wizard's range is always 2.

Methods

Name	Description
+ int attack(Unit e)	If the target is in the same team as the wizard or the target's location is out of range, return -1 to signify a failed attack. Otherwise, return the wizard's power directly. Hint: BattleUtils class in the logic package should contain useful static methods for this.

4.2.3 public class **Guildmaster**

This class is a unit that can do both melee attacks and can raise a guard (but never does so), and is controlled by a simple AI.

Variables

None.

Constructor

Name	Description
+ Guildmaster(int maxHealth, int speed, int power, int defense, int location)	Initializes the values for the guildmaster. For the values not given in the constructor, they are fixed as follow: Guildmaster's name is always "Guildmaster". Guildmaster's symbol is always "G" (uppercase). Guildmaster's playerControlled is always false. Guildmaster's range is always 1.

Methods

Name	Description
+ boolean move(int spaces)	No matter what number is inputted, the Guildmaster always attempts to move -1 spaces. (One to the left)

+ void guard()	Since the Guildmaster's AI never guards, this function does nothing.
+ int attack(Unit e)	If the target is in the same team as the guildmaster or the target's location is out of range, return -1 to signify a failed attack. Otherwise, return the damage after having deducted the target's defense (if they have any). Hint: BattleUtils class in the logic package should contain useful static methods for this.

4.3 package logic

ALREADY GIVEN

Everything in this package is already written for you. Only important variables and methods will be listed for you to use.

4.3.1 public class BattleUtils

Variables

None.

Methods

Name	Description
+ int calculateDamage(int power, Unit e)	Static method. If unit e is an instance of Guardable, then e's defense is saved as an int to deduct from the attacker's power. Then, if e is on guard, the deduction is doubled.

	Finally, subtract the final deduction from the power, and return the damage. If it is less than 0, then return 0.
+ boolean validRange(int range, int myLocation, int targetLocation)	Static method. If the absolute of the sum between myLocation and targetLocation is lesser than or equal to range, then it returns true. Otherwise return false.

5. Scoring Criteria

Your work will be run through JUnit test cases similar to the ones you have been provided, and your scores will be based on how many test cases you passed, as well as whether or not you have submitted a correct UML Diagram.

This part is worth a total of 31 points, rounded to 10.

5.1 TankTest (Worth 9 points)

- testTankConstructor (2)
- testTankTakeDamage (1)
- testTankGuard (1)
- testTankGuardBreakFromMoving (1)
- testTankNoAttack (2)
- testTankGuardable (2)

5.2 WizardTest (Worth 9 points)

- testWizardConstructor (2)
- testWizardDealDamage (1)

- testWizardNoFriendlyFire (1)
- testWizardRange (1)
- testWizardNoGuard (2)
- testWizardAttackable (2)

5.3 GuildmasterTest (Worth 11 points)

- testGuildmasterConstructor (2)
- testGuildmasterTakeDamage (1)
- testGuildmasterGuard (1)
- testGuildmasterNoFriendlyFire (1)
- testGuildmasterRange (1)
- testGuildmasterMove (1)
- testGuildmasterAttackable (2)
- testGuildmasterGuardable (2)

5.4 Correct UML Diagram (Worth 2 points)